Master The MQTT

MQTT is an OASIS standard messaging protocol for the Internet of Things (IoT). It is designed as an extremely lightweight publish/subscribe messaging transport that is ideal for connecting remote devices with a small code footprint and minimal network bandwidth.

Why MQTT?

Lightweight and Efficient

MQTT clients are very small, require minimal resources so can be used on small microcontrollers. MQTT message headers are small to optimize network bandwidth.

Bi-directional Communications

MQTT allows for messaging between device to cloud and cloud to device. This makes for easy broadcasting messages to groups of things.

Scale to Millions of Things

MQTT can scale to connect with millions of IoT devices.

Reliable Message Delivery

Reliability of message delivery is important for many IoT use cases. This is why MQTT has 3 defined quality of service levels: 0 - at most once, 1- at least once, 2 - exactly once

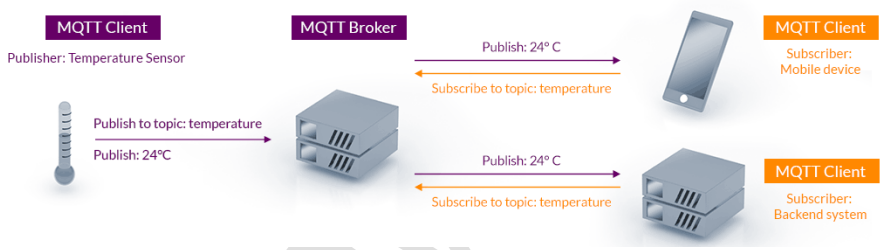
Support for Unreliable Networks

Many IoT devices connect over unreliable cellular networks. MQTT's support for persistent sessions reduces the time to reconnect the client with the broker.

Security Enabled

MQTT makes it easy to encrypt messages using TLS and authenticate clients using modern authentication protocols, such as OAuth.

MQTT Publish / Subscribe Architecture



Detailed End-to-End Flow:

1. **1. Establish Connection:**

A publisher and subscriber both establish a network connection with the MQTT broker.

2. **2. Publishing:**

The publisher sends a message, including the topic it's related to, to the broker.

3. **3. Broker's Role:**

The broker receives the message and uses the topic to identify which subscribers are interested in receiving it.

4. **4. Subscription Management:**

The broker maintains a list of subscribed topics and the corresponding subscribers.

5. **5. Delivery to Subscribers:**

The broker delivers the message to all subscribers who are interested in that specific topic.

6. **6. Asynchronous Communication:**

MQTT allows for asynchronous communication, where the publisher doesn't need to wait for confirmation from the broker or subscribers.

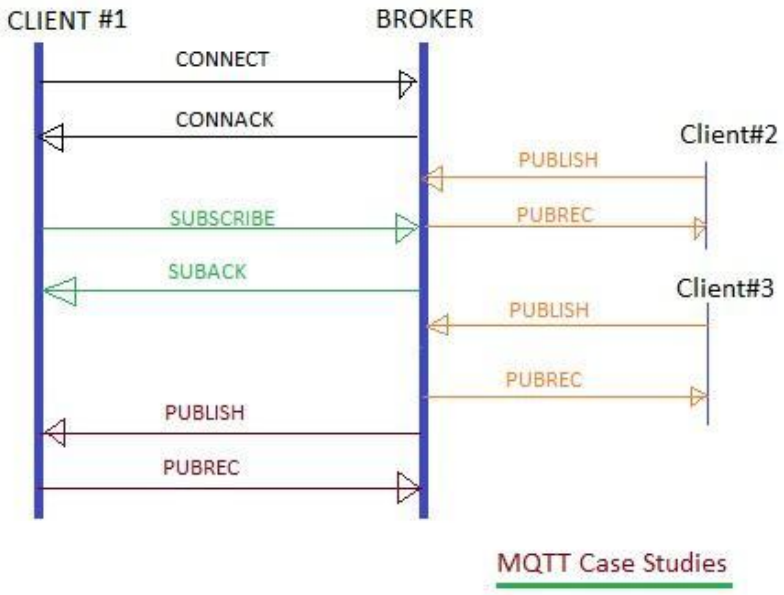
7. **7. Message Acknowledgments:**

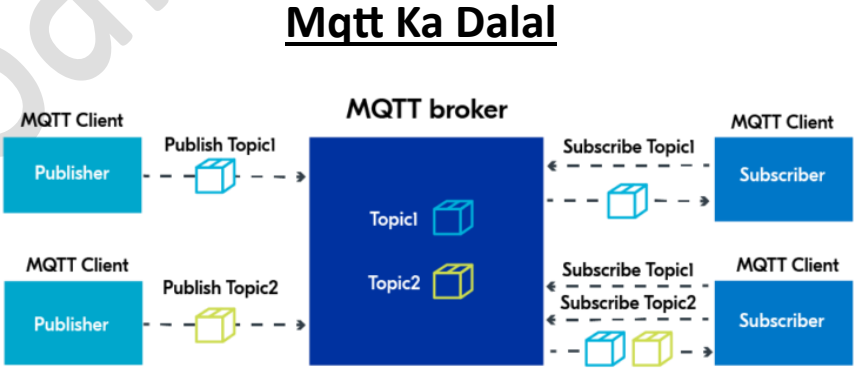
MQTT provides mechanisms for ensuring message delivery, including acknowledgment messages (e.g., PUBACK, SUBACK).

8. **8. Security:**

MQTT can be secured using various methods like SSL/TLS, certificate-based authentication, or username/password authentication.

MQTT Architecture Working: Example Use Cases





An MQTT broker is just a server that receives messages from connected clients and routes them to the applicable destination clients. Multiple clients can be subscribed to a single topic and a single client can be subscribed to topics with the broker.

MQTT works on ***publisher/subscriber pattern*** in which two clients (publishers or subscribers) never contact directly and instead communication between them is handled by a third party. In MQTT, we call this third-party "Broker".

Aspects of MQTT protocol

- To communicate with each other, publisher and subscriber only needs to know IP and port of the broker and a common **Topic**. This removes dependency of each client knowing address of other client.
- Even though most messages are delivered on the go, MQTT also supports storing of messages. If desired, broker can store these messages when a client is not online. For this client needs to connect using a persistent session with ***QOS*** greater than zero.
- MQTT mostly works in synchronous manner which results in no lagging in delivery.
- MQTT uses subject based filtering and broker uses ***Topics*** to decide on which subscriber is to receive which message.

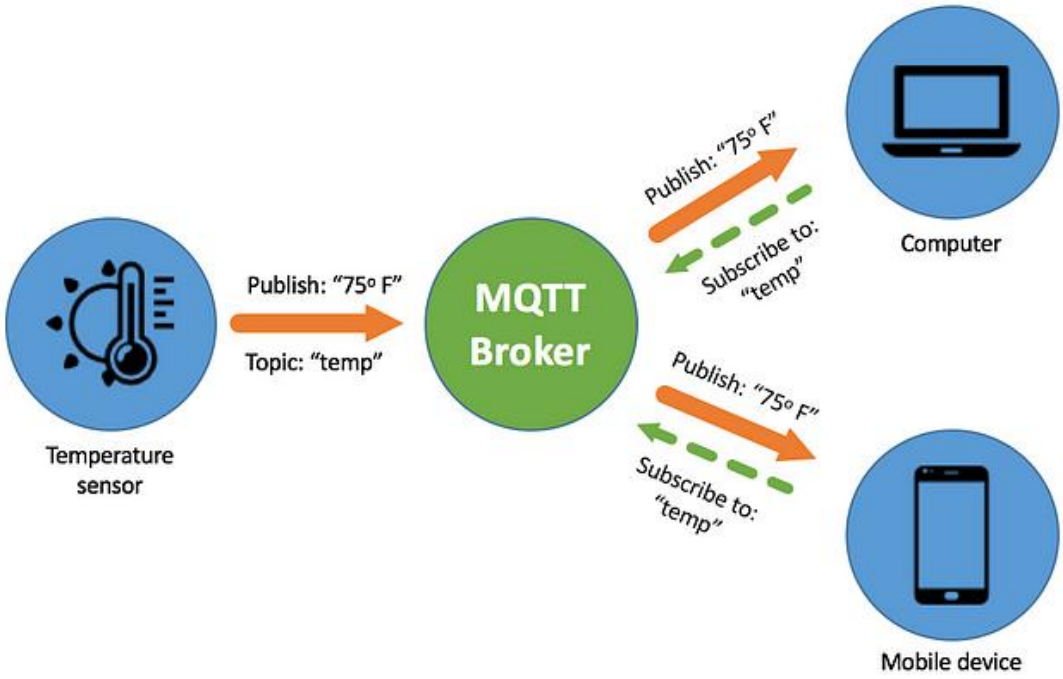


Figure : MQTT in temperature sensing.

Temperature Sensing example with MQTT Implementation

A temperature sensor publishes real-time temperature values to "temp" topic on MQTT-Broker.

Broker then multicasts this information to all devices that had subscribed to topic "temp".

MQTT - Client

- In MQTT, both publishers and subscribers are clients.

- A client can act both as a publisher or subscriber and name only signifies sending or receiving party.
- Ease of implementation and MQTT-client's ability to work on low bandwidth network is what makes it perfect to use in IOT based devices.
- Micro-controllers, pi, AWS servers running on MQTT can act as clients.
- Different languages based client libraries are available.

MQTT - Broker

- Broker acts as a mediator between publisher and subscriber.
- MQTT brokers can handle thousands of messages at a time due to its simplified architecture. Here is a comparison between **MQTT and HTTP protocols**.
- Work done by broker : Receive all messages, **filter** these messages, find out subscribers to a particular topic, Send these message to those subscribed clients.
- Broker can also hold missed messages of persisted clients which can be sent to it once he comes online again. This is important for a case when some client gets disconnected unwillingly.
- Broker is also responsible for handling authentication and authorization.

MQTT - Connection

- MQTT broker and clients both are TCP/IP based.
- Connection is always between one broker and one client.
- Some important parameters needed for the connection :

*"**clientId**" : should be unique per client and is needed to maintain state of the client. This will help to deliver new messages to the client which were not delivered to it when it was offline.*

*"**cleanSession**" : equals to false, if you want to establish persistent session and store these subscriptions.*

*"**username/password**" : sent as plain text if not encrypted.*

*"**willMessage**" : message to be sent by the disconnecting client to all clients of that topic in case it gets disconnected.*

Need for Client-ID : Client-ID is needed to identify an application mainly when a client is using persistent subscriptions. When these subscriptions are offline, broker creates a backup of messages been delivered to them and when such applications come on-line again, a messaging provider has to identify these subscriptions and there messages. Broker then compares the persisted client-id across clients that came online and delivers messages to them.

Publisher/Subscriber in MQTT

- A MQTT **Publisher** client can start publishing messages as soon as it connects to the broker. Each message will contain a topic and payload to be sent.
- A MQTT **Subscriber** are the ones that receives messages that are published to a particular topic.

MQTT - Topics

- MQTT-Topic is a string that broker uses to filter messages. Only subscribers of a particular topic receives messages published to that topic.
- Topics consist of topic levels separated by a "/". Example : *"home/ground/room/temp"*
- Properties :
  1. Must contain atleast 1 character
  2. Can contain empty spaces
  3. "/" is a valid topic name

- **Quality of Service** : ***QOS*** is a key feature of the MQTT protocol which gives client a power to choose among different level of services that matches its network reliability and application logic. Because MQTT manages the re-transmission of messages and guarantees delivery (even when the underlying transport is not reliable), QOS makes communication in unreliable networks a lot easier.
- **Persistent Session in MQTT** : In MQTT a client can request a persistent session when it connects to the broker. Persistent sessions save all information that is relevant for the client on the broker.